

Omega TK Intelligent Code Assistant

A Production-Deployed RAG Architecture for Domain-Specific Python Generation

`Status: v2.0 Live | Architecture: Retrieval-Augmented Generation | Stack: FastAPI + React + FAISS`

The Engineering Whitepaper: Architectural Comparison for Computational Chemistry API Task Automation

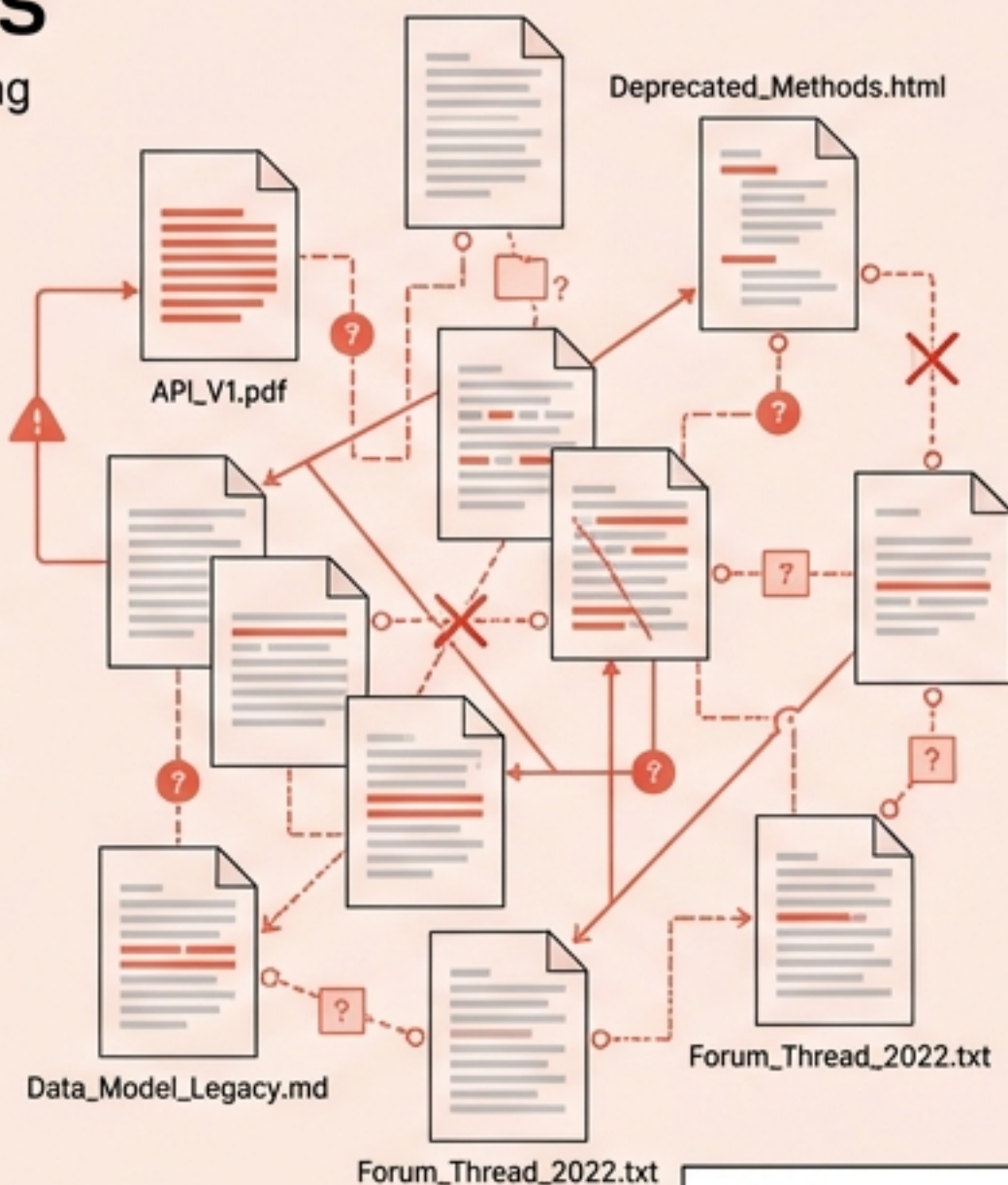
THE PROBLEM (FRAGMENTATION & HALLUCINATION)

65 Minutes

Manual task time navigating dense documentation

60% Failure Rate

GPT-4 hallucination rate without domain grounding

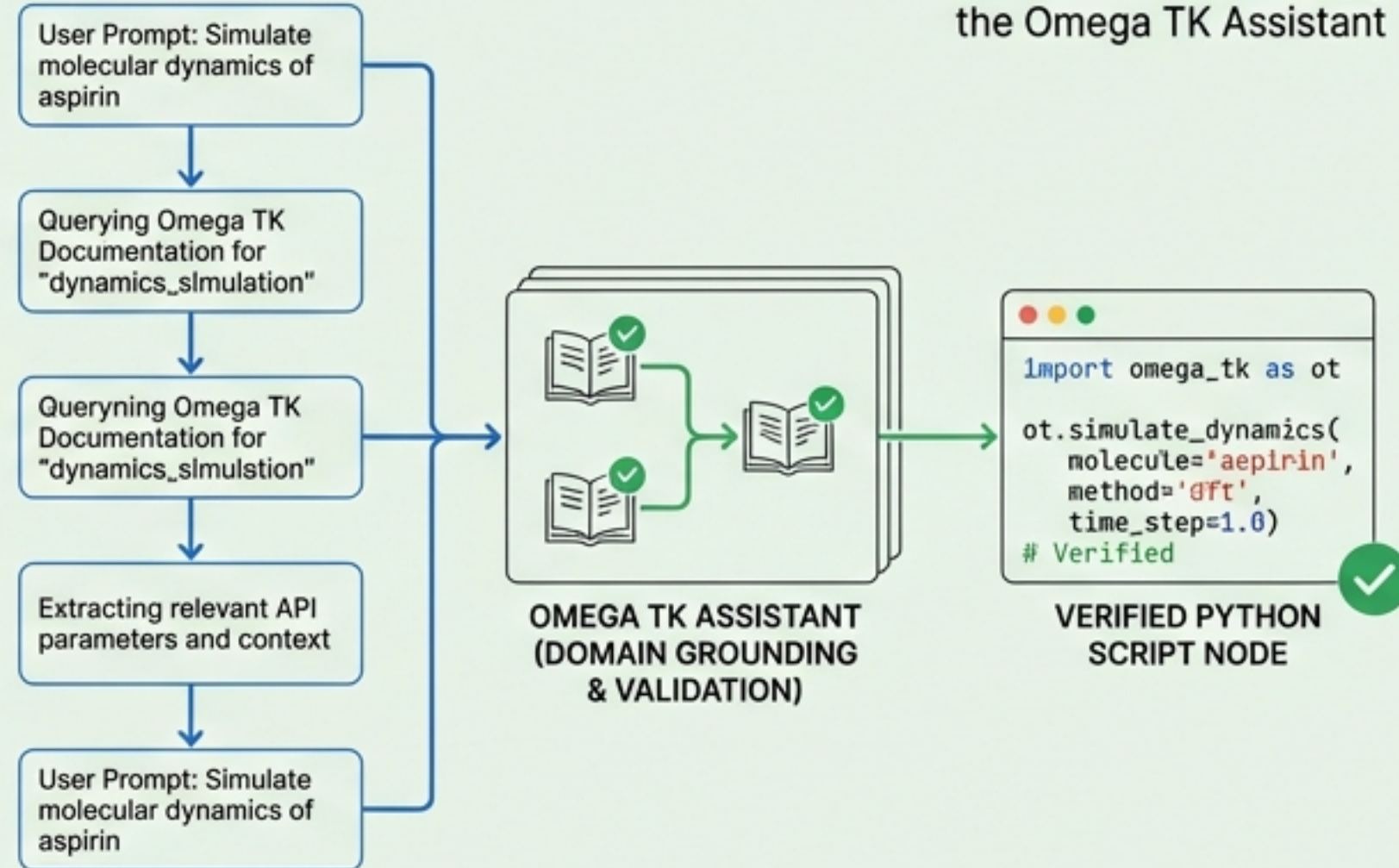


Solving hallucination in niche computational chemistry APIs requires grounding generative AI in actual documentation.

THE SOLUTION (GROUNDED AUTOMATION with OMEGA TK)

< 5 Minutes

Task completion time using the Omega TK Assistant



The Engineering Whitepaper:

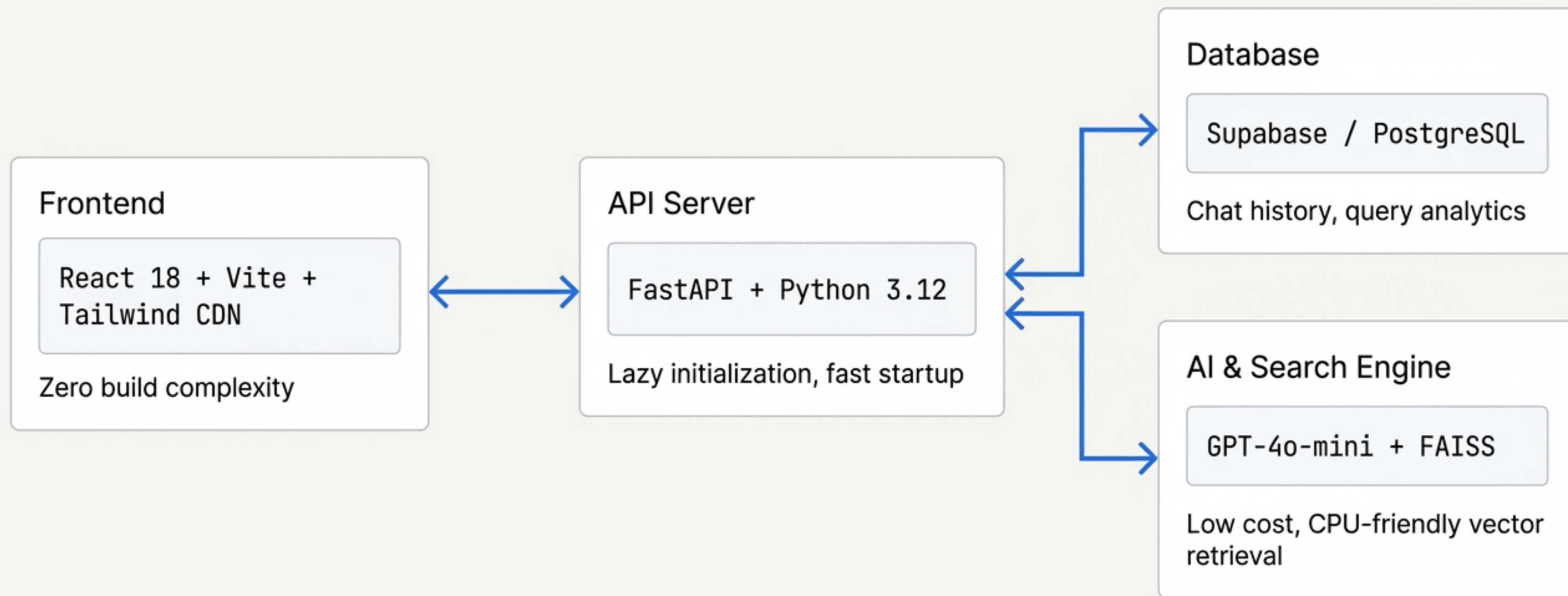
Architectural Comparison of API Automation Approaches

Approach	Hallucination Risk	Compute Cost	Updatability
Keyword Search	N/A	Low	Fails on paraphrased queries
Pure LLMs (GPT/Claude)	High (on niche APIs)	Medium	Requires prompt engineering
Fine-Tuned Model	Low	High (Requires costly GPU compute)	Needs thousands of labeled examples to update
Selected Architecture			
RAG (Omega TK Approach)	Near Zero	Low	Updates via simple document indexing

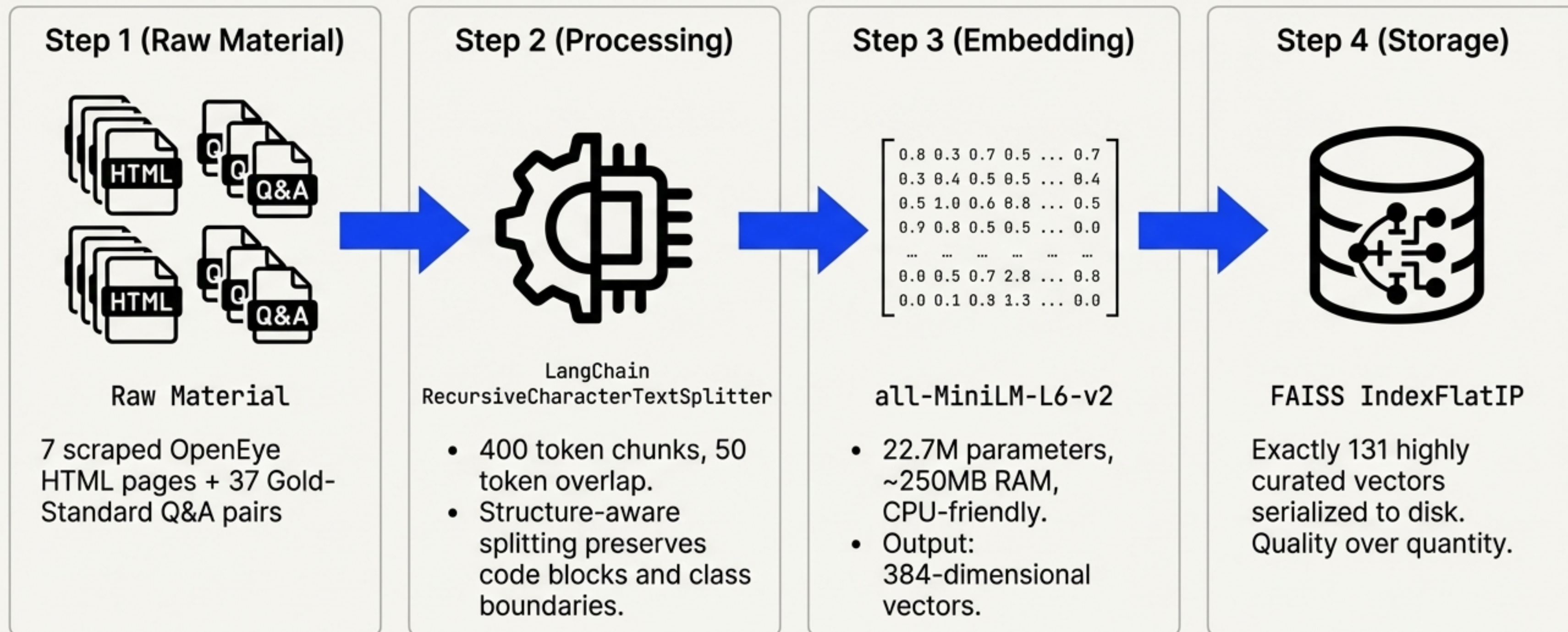
Key Insight: RAG provides the optimal balance. Adding new documentation pages requires zero retraining, and hallucination is reduced to near zero for in-scope queries.

The Engineering Whitepaper: Tech Stack Macro Architecture Blueprint

`Key Design Principle: Zero Budget`
Every tool utilized is free-tier or covered by existing API credits.

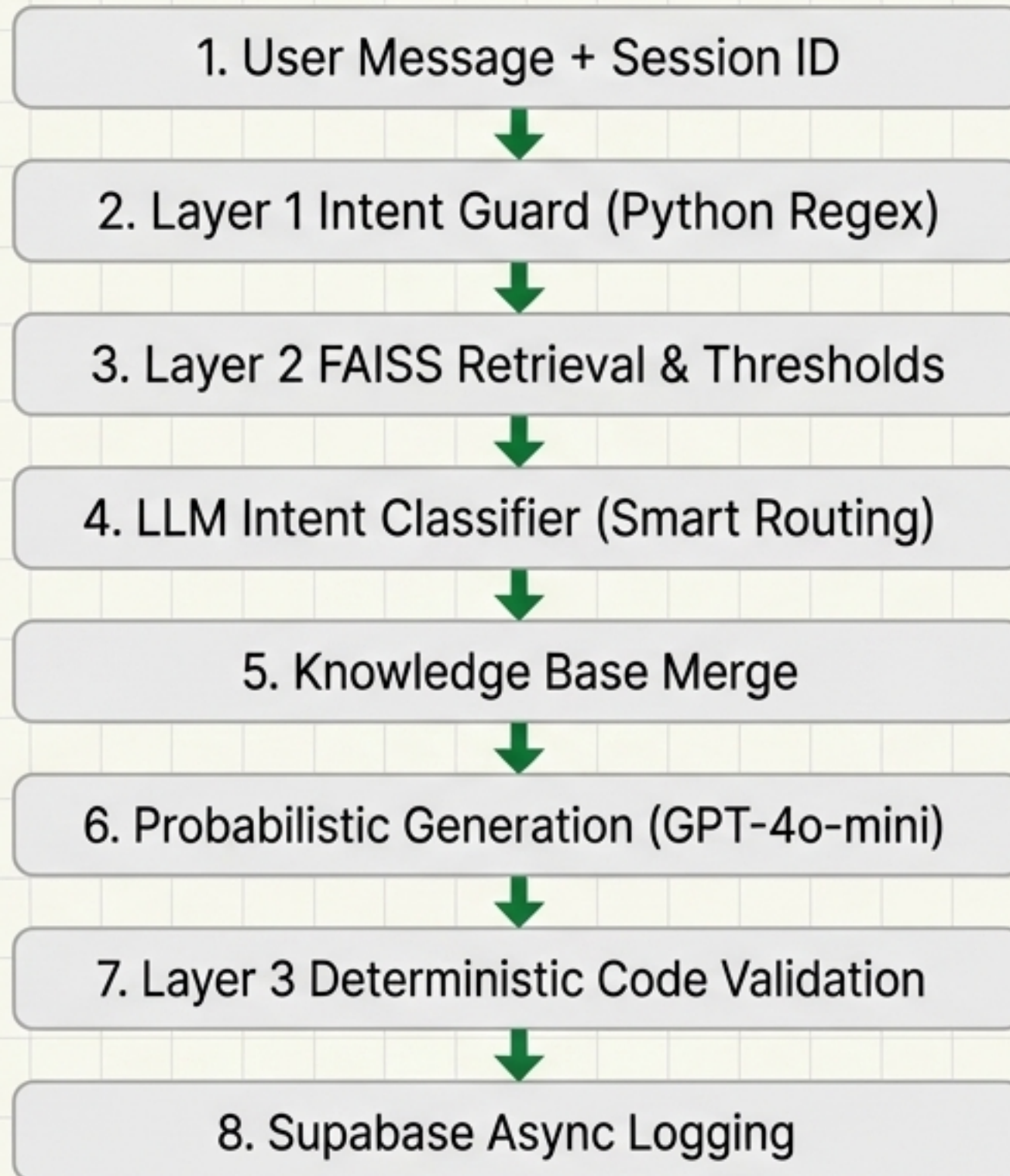


The Engineering Whitepaper: Data Factory Funnel - Offline Knowledge Preparation



The Engineering Whitepaper:

Runtime Request Pipeline - End-to-End Flow



Every request follows this exact pipeline. 59 automated pytest integration tests ensure zero regressions across this precise flow.

The Engineering Whitepaper: Pre-LLM Defensive Filtering

⚙️ Layer 1: Intent Guard

Pure Python keyword check. Runs in <1ms. Zero API cost.

Blocklist (INVALID signals)

ignore
forget previous
malicious_code
system prompt injection
jailbreak

Triggers instant fallback.

Allowlist (VALID signals)

explain
how to
documentation
api usage
best practices

Passes through.

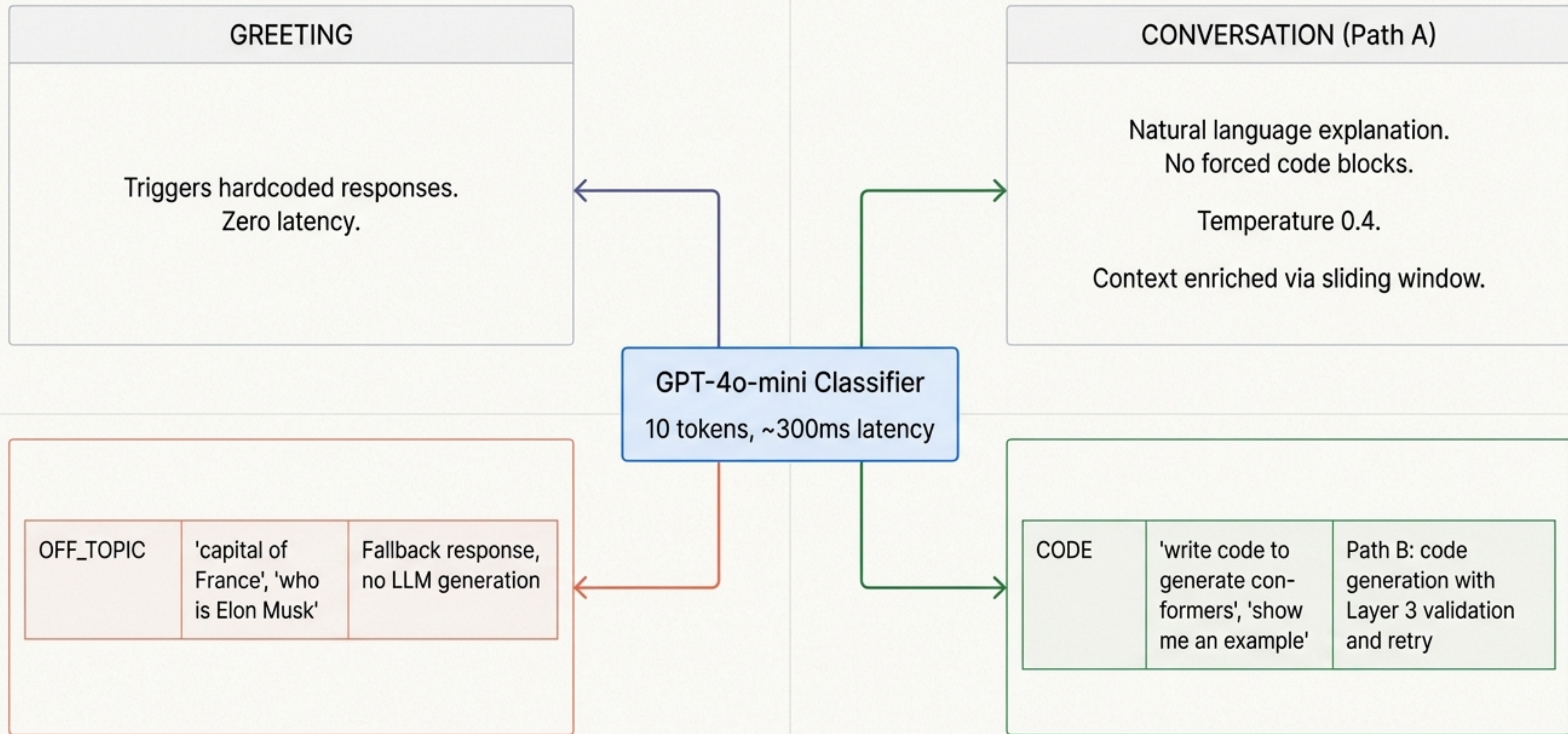


🗄️ Layer 2: FAISS Retrieval Confidence

Cosine similarity checks against the 131-vector index.

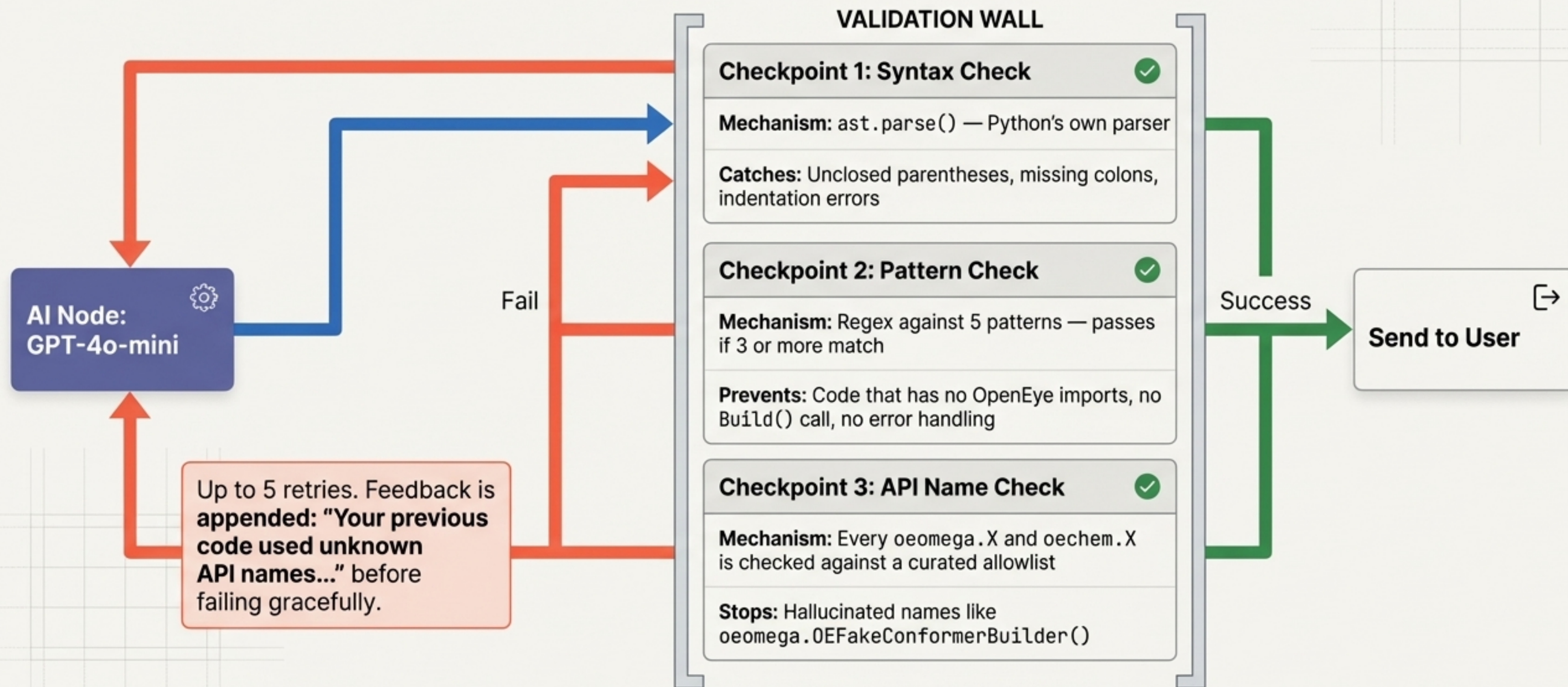
Concept Queries	↓	0.20 threshold	(accommodates natural language)
Code Queries	↓	0.30 threshold	(stricter relevance)

Action: Falling below threshold triggers an "I don't have enough information" response, eliminating ungrounded hallucinations.



Context-aware routing includes the last two turns of history, ensuring prompts like 'verify this code' correctly route to Conversation, not Code.

The Engineering Whitepaper: Probabilistic Generation + Deterministic Validation Retry Loop



The Anatomy of an OpenEye Code Block

```
from openeye import oechem, oeomega

ifs = oechem.oemolistream('in.sdf')
opts = oeomega.OE0megaOptions()
omega = oeomega.OE0mega(opts)
mol = oechem.OEMol()
while oechem.OEReadMolecule(ifs, mol):
    ret_code = omega.Build(mol)
    if ret_code == oeomega.OE0megaReturnCode_Success:
        print(f"Successfully built conformers for {mol.GetTitle()}")
    else:
        print(f"Failed to build for {mol.GetTitle()}:
              {oeomega.OE0megaGetReturnCodeString(ret_code)}")
ifs.close()
```

1. Imports

Load the OpenEye modules

`from openeye import oechem, oeomega`

2. Streams

Open input/output molecule files

`ifs = oechem.oemolistream('in.sdf')`

3. Options

Configure the Omega settings

`opts = oeomega.OE0megaOptions()`

4. Build

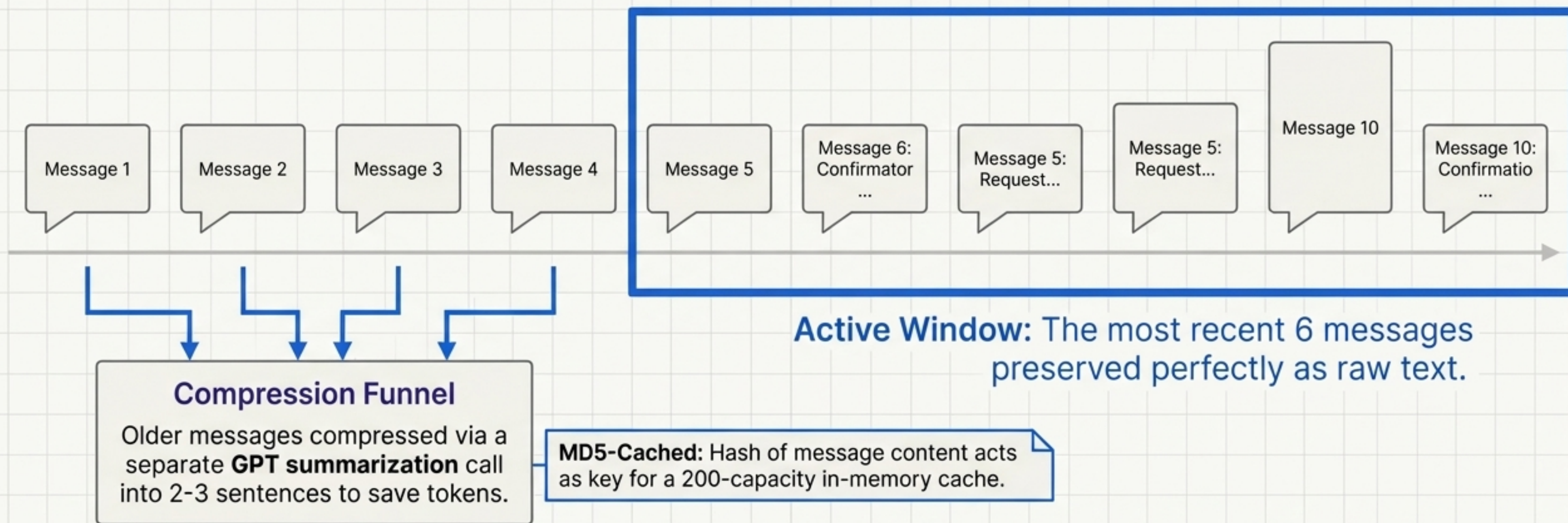
Build: ``ret_code = omega.Build(mol)``

5. Error Check

Error Check: ``OE0megaReturnCode_Success``

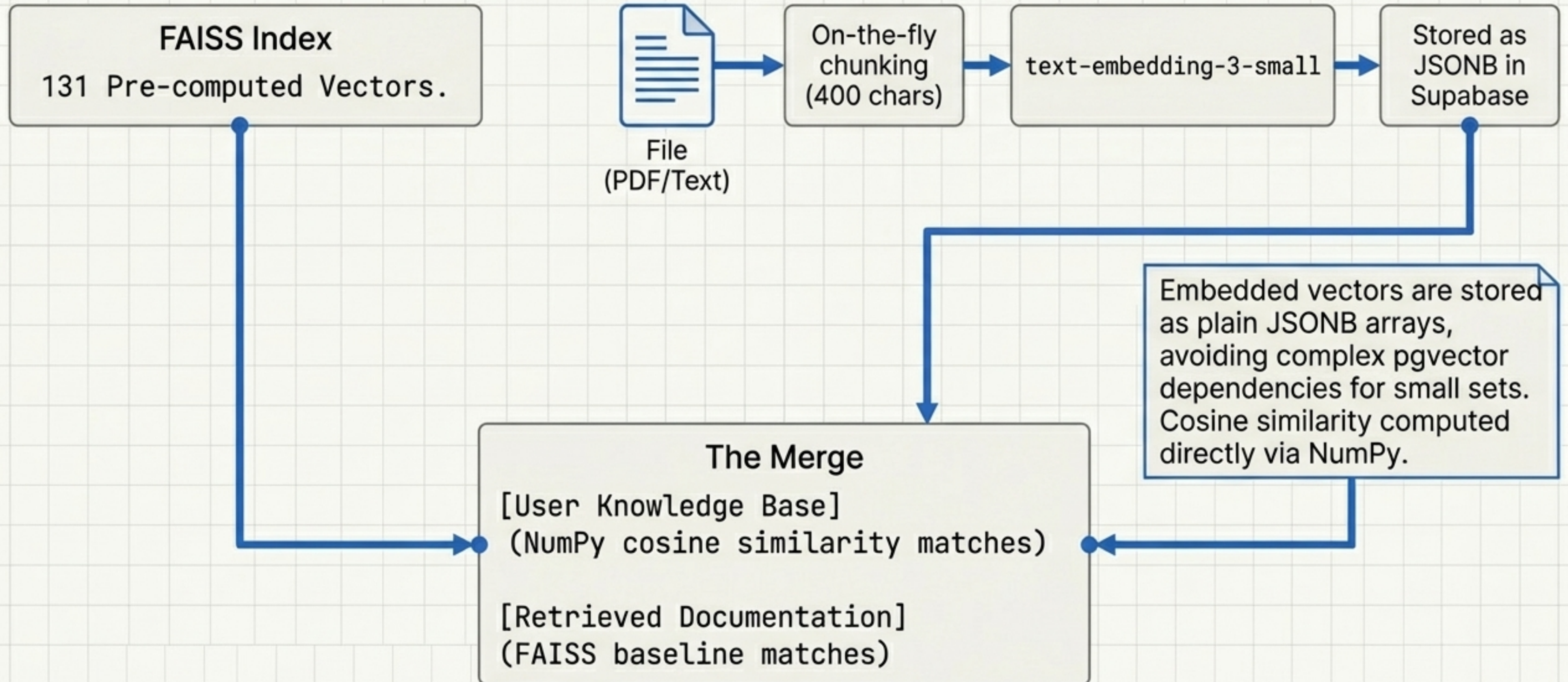
The validator checks that all five structural components are present before accepting the LLM output, guaranteeing domain accuracy.

The Engineering Whitepaper: Conversation Memory Management Strategy



Query Enrichment: Follow-up signals (e.g., 'now use dense sampling') automatically trigger **appending prior messages to the query**, boosting **FAISS** retrieval scores from ↗ 0.26 to ↗ 0.61.

The Engineering Whitepaper: On-the-Fly Document Merge Strategy



The Engineering Whitepaper: Persistence and Resilience

Supabase Database Schema

Table	Key Columns	Purpose
chat_history	session_id, role, content	Enables UI session restore
queries_log	intent, faiss_score, latency, attempts	Powers analytics dashboard
feedback	thumbs up/down	Tracks response quality
knowledge_chunks	stores user JSONB embeddings	Stores user JSONB embeddings

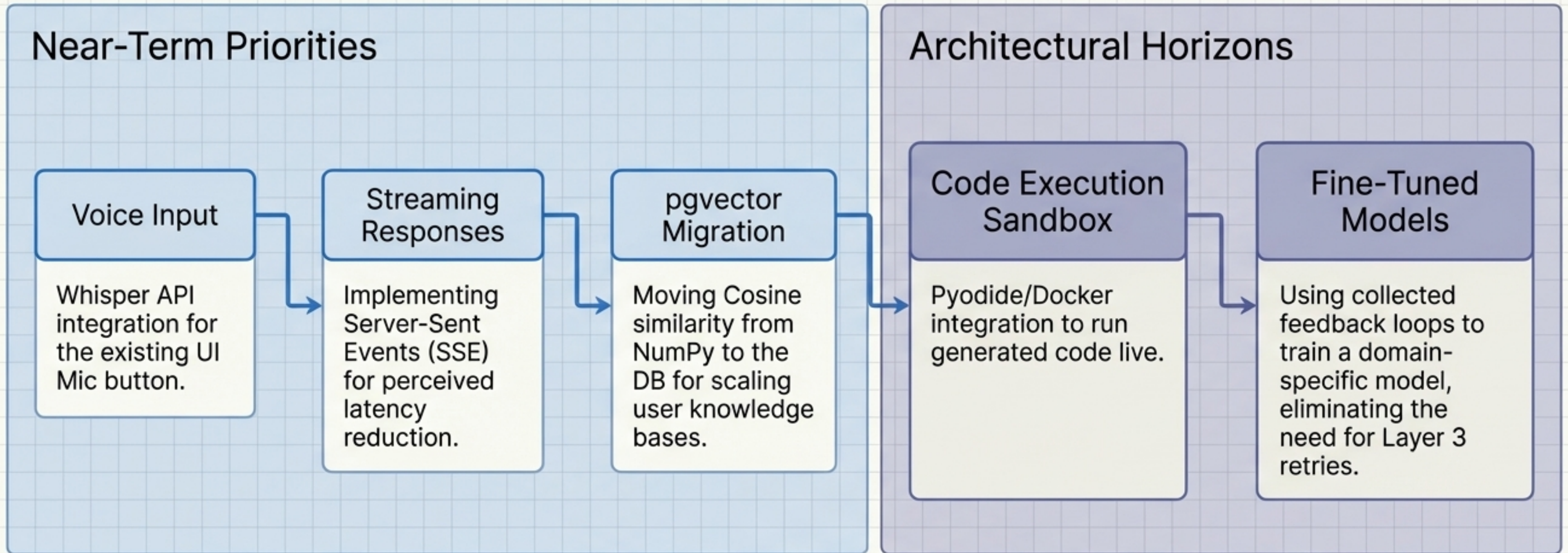
System Resilience



If database connections fail, reads return empty lists and writes fail silently. The core chat interface and OpenAI logic continue to function perfectly.

Local development only requires an OpenAI key.

The Engineering Whitepaper: Product Roadmap and Architectural Vision



Standard Python software engineering transforms volatile LLMs into deterministic, production-ready tools.